

Transformer WAF + Robust Phishing URL Detection — Project Handover

Everything needed to run, understand, demonstrate and extend the project

September 2026

1. What this project is

A Transformer-based Web Application Firewall (WAF) that was extended into a research system for the question:

How does missing feature information affect the ability of a machine-learning model to detect phishing URLs, and can an appropriate missing-information handling strategy enable the model to continue detecting phishing URLs reliably?

It is one web application (FastAPI, port 8000) with two Transformer models:

Component	Trained how	Answers
WAF anomaly model (original)	Unsupervised, on benign HTTP requests only	“Is this request unlike normal traffic?” (SQL injection, XSS, traversal...)
Phishing URL classifier (new)	Supervised, on 164,754 labeled URLs (PhiUSIIL)	“Is this URL phishing or legitimate?” — in two variants: <i>baseline</i> and <i>robust</i>

Plus a **research module** that removes parts of a URL in a controlled way, compares three handling strategies, computes metrics, stores every run in SQLite and shows it in the UI.

Headline result (measured on all 47,072 held-out test URLs):

Missing information	No handling	[MISSING] marker only	Mitigation (model trained with incomplete examples)
0 %	99.80 %	99.80 %	99.81 %

Missing information	No handling	[MISSING] marker only	Mitigation (model trained with incomplete examples)
10 %	92.63 %	87.79 %	98.58 %
30 %	79.71 %	68.54 %	95.16 %
50 %	69.14 %	55.03 %	89.81 %

(accuracy; full precision/recall/F1/FPR/FNR/AUC tables are in `README.md` §8 and on the Research page.)

2. What is in the box

```

transformer-waf/
├─ README.md          full technical documentation (start here after this
file)
├─ docs/
│   ├─ HANDOVER.md    this document
│   ├─ DEMO_SCRIPT.md 5-minute demo / video script, click by click
│   └─ EXAMPLES.md    verified URLs to test with, and what to expect
├─ requirements.txt    Python dependencies
├─ config.yaml         WAF settings (threshold, model dims, ports)
├─ scripts/
│   ├─ setup_and_run.sh / .bat ONE COMMAND: venv + install + start service
│   ├─ package_release.sh      builds the hand-over zip (models + dataset included)
│   ├─ prepare_waf_data.py     WAF training data + vocabulary from data/training/
│   ├─ train_quick.py          trains the WAF anomaly model
│   └─ calibrate_threshold.py  measures TPR/FPR and writes the threshold to
config.yaml
├─ tooling
│   ├─ fetch_phishing_dataset.py downloads PhiUSIIL -> data/phishing/urls.csv
│   ├─ train_url_classifier.py  trains baseline + robust URL classifiers
│   ├─ run_robustness_experiments.py runs the experiments from the terminal
│   └─ run_tests.sh, eval_benchmark.py, demo/start/stop scripts (original WAF
tooling)
├─ src/
│   ├─ api/detection_api.py    the FastAPI app: WAF endpoints + UI routes, includes
↓
│   ├─ api/phishing_routes.py  research endpoints (/detect/url, /experiment/...)
│   ├─ api/static/             the web UI (landing, analyze, research, history)
│   └─ preprocessing/         tokenizer (with [MISSING] token), request composer,
parser
│   ├─ models/                WAFTransformer, TransformerURLClassifier, inference
│   └─ phishing/              url_features, missingness, dataset, training,
metrics,
│   └─ experiments, service   (the research module)
└─ storage/                   SQLite stores: detections, URL analyses, experiments

```

```

├── utils/                                config + logging
├── data/
│   ├── training/*.jsonl                raw benign/attack HTTP requests (WAF)
│   ├── train/                          encoded WAF training set + held-out calibration sets
│   └── phishing/urls.csv                235,362 labeled URLs with train/val/test split (in
the zip; not in Git)
├── models/
│   ├── checkpoints/best.pt + vocab.json  WAF model (in the zip; not in Git)
│   └── phishing/url_classifier_{baseline,robust}.pt + vocab.json + *.json metadata
├── logs/detections.db                  SQLite: detection history, URL analyses, stored
experiments
├── tests/                             51 automated tests (pytest)
├── integration/, sample_apps/, Dockerfile, docker-compose.yml  original WAF
deployment helpers

```

3. Running it

3.1 From the release zip (recommended — nothing to train)

Requirements: Python 3.11 or newer, internet access for `pip` the first time.

```

unzip transformer-waf-release-YYYYMMDD.zip
cd transformer-waf
./scripts/setup_and_run.sh          # macOS / Linux
scripts\setup_and_run.bat          # Windows

```

Then open **`http://localhost:8000`**. The API key for every request is `dev-key` (already filled in the top-right box of the UI). Stop with `Ctrl+C`; start again with the same command (the second start takes seconds).

3.2 From Git (models must be built first)

Same script: it detects that the models/dataset are missing and builds them (WAF $\approx$ 3 min, dataset download $\approx$ 15 MB, URL classifiers $\approx$ 20 min on CPU). The individual commands are in `README.md` §2–§5.

3.3 Pages

URL	Page	What it does
/	Overview	Landing page: hero with the security visual and a live WAF terminal, live metrics, 4-step research method, live

URL	Page	What it does
		complete/missing/mitigated analysis, stored research results and chart, components, CTA
<code>/analyze</code>	Analyze	URL classifier with missing-information controls, information-availability bar, Run experiment (complete → missing → mitigated story) and Missing-level tests (0/10/30/50 % + mitigation), examples; WAF request scoring
<code>/research</code>	Research	Tabs: research overview (problem, dataset, model, method, conclusion), missing-rate sweep, feature dependency, Results (<code>#results</code>), stored runs
<code>/history</code>	History	Everything stored in SQLite: WAF requests (with replay) and URL analyses
<code>/docs</code>	API	Interactive Swagger documentation of every endpoint
<code>/documentation</code>	Documentation	This hand-over document as PDF

The sun icon in the top bar switches between the dark (default) and light theme.

4. How it works (the 10-minute version)

4.1 The WAF anomaly model

An HTTP request is composed into one string — `GET /search?q=<script>... BODY:...` — tokenised on punctuation, and fed to a Transformer trained to predict each next token of *benign* requests. The **anomaly score** is the mean next-token negative log-likelihood: high means “this is not like normal traffic”. The threshold in `config.yaml` (3.06) was calibrated on held-out data: 97 % of attack requests above it, 0.9 % of benign requests above it.

4.2 The phishing URL classifier

The *same* Transformer encoder (`WAFTransformer.encode`) with a small classification head, trained with binary cross-entropy on labeled URLs (1 = phishing). Two checkpoints:

- **baseline** — trained on complete URLs.

- **robust** — trained with *missing-information augmentation*: every epoch, each training URL has a random subset of its segments replaced by the [MISSING] token. This is the mitigation strategy under test.

4.3 What “missing information” means here

The model reads a token sequence, not a table of numbers, so the units of information that can be unavailable are the **segments** of the URL:

scheme · subdomain · domain · tld · path · query · fragment

Example, with scheme and domain unavailable:

original	https://login.secure-paypal.co.uk/account/verify.php?id=1
blank (none)	login.co.uk/account/verify.php?id=1
[MISSING] token	[MISSING]://login.[MISSING].co.uk/account/verify.php?id=1

The segment is removed *before* tokenisation, so nothing downstream can recover it (no information leak).

4.4 The three handling strategies

Strategy (UI label)	What happens to a missing segment	Model used
None (blank the segment)	dropped as if never extracted	baseline
Explicit [MISSING] marker	replaced by the [MISSING] token	baseline
Training with incomplete examples	[MISSING] token	robust (the mitigation)

4.5 The experiments

- **Missing-information sweep** — each segment of each test URL is unavailable with probability $p \in \{0, 10, 20, 30, 40, 50 \%\}$; optionally some segments are always unavailable (single/multi-feature conditions). All strategies see the *same* masked inputs (same seed).
- **Feature dependency** — each segment removed on its own (plus optional combinations) to measure which information the model relies on.

Metrics per condition: accuracy, precision, recall, F1, FPR, FNR, ROC AUC, confusion matrix, degradation vs. complete information, recovery vs. no handling. Every run is stored with its seed, sample size, dataset hash and model provenance, so it can be reproduced exactly.

4.6 What the measurements showed

1. Missing information hurts a lot: 99.80 % → 69.14 % accuracy at 50 % missing with no handling, mostly through false positives (FPR 0 % → 42 %).
2. An explicit marker *without* retraining is worse than blanking — the baseline model never saw the marker.
3. Training with incomplete examples recovers most of the loss (89.81 % at 50 % missing, 98.58 % at 10 %) at no cost when information is complete (99.81 % vs 99.80 %).
4. The baseline depends on two shallow cues: `subdomain` (mostly `www`) and `scheme` (`https`). Reason: in PhiUSIIL 100 % of legitimate URLs are `https`, 93.7 % start with `www` and **0 % have a path**, while phishing URLs are 48.5 % / 30.1 % / 59.7 %. The robust model no longer needs `subdomain`.

5. Known limitations (say these out loud in a viva)

- **Dataset artifact.** Because no legitimate URL in PhiUSIIL has a path, *any* URL with a path is classified as phishing — including `https://www.github.com/login`. This is a property of the training data, not a bug; the pipeline accepts any other `url,label` CSV (`scripts/fetch_phishing_dataset.py --from-csv ...`) and would learn differently.
- **Segments, not page features.** Page-content/WHOIS features used by tabular phishing detectors do not exist in a sequence model and are not simulated.
- **The WAF model is unsupervised.** It flags unusual requests; it does not classify phishing. Headers are not part of its input; very long bodies collapse to one unknown token.
- Single-process SQLite; experiments run synchronously (5,000 URLs × 6 rates × 3 strategies takes a few seconds; the full 47,072-URL split about 5 minutes).

6. Retraining and reproducing

```
# WAF anomaly model
PYTHONPATH=. python scripts/prepare_waf_data.py
PYTHONPATH=. python scripts/train_quick.py --epochs 40 --batch 64 --vocab 5000 --embed
128 --heads 4 --layers 3 --ff 256 --maxlen 128 --lr 5e-4
PYTHONPATH=. python scripts/calibrate_threshold.py --benign-percentile 0.99 --write

# URL classifiers (both variants; 3 epochs each, seed 42)
PYTHONPATH=. python scripts/train_url_classifier.py --variant both --epochs 3 --
rebuild-vocab
curl -X POST localhost:8000/url/reload -H "X-API-Key: dev-key"      # if the service
is running

# Experiments (reproduce the README tables)
```

```
PYTHONPATH=. python scripts/run_robustness_experiments.py --sample-size 0 --seed 42 --out results.json
```

Activate the virtual environment first (`source venv/bin/activate` or `venv\Scripts\activate`).

7. Tests

```
PYTHONPATH=. python -m pytest -q tests          # 51 tests, ~3 s
bash scripts/run_tests.sh                      # legacy live-server check (service
must be running)
```

8. API quick reference (all need header `X-API-Key: dev-key`)

Method	Path	Purpose
POST	<code>/detect</code>	Score one HTTP request with the WAF model
POST	<code>/detect/batch</code>	Score many requests
GET	<code>/logs</code> , <code>/stats</code> , POST <code>/replay/{id}</code> , POST <code>/threshold</code>	WAF history, statistics, replay, threshold
POST	<code>/detect/url</code>	Classify a URL; body <code>{url, missing_features[], strategy, persist}</code>
GET	<code>/url/status</code>	Models, dataset, features, strategies available
GET	<code>/url/history</code>	Stored URL analyses
POST	<code>/experiment/missing-information</code>	<code>{rates[], features[], strategies[], sample_size, seed}</code>
POST	<code>/experiment/feature-impact</code>	<code>{features[], combinations[], strategies[], sample_size, seed}</code>
GET	<code>/experiment/results[/]{id}</code>	Stored experiments (DELETE removes one)

Example:

```
curl -s -X POST localhost:8000/detect/url -H "X-API-Key: dev-key" -H "Content-Type: application/json" \
  -d '{"url":"https://www.readersdigest.co.uk","missing_features":
["subdomain","scheme"],"strategy":"augmented_training"}'
```

9. Troubleshooting

Symptom	Cause / fix
Page says “model missing” / analyzer disabled	<code>models/phishing/*.pt</code> absent — run <code>scripts/train_url_classifier.py</code> or use the release zip
“Cannot reach the API” in the UI	service not running; start it with <code>scripts/setup_and_run.sh</code>
401 Unauthorized	API key box in the top bar must contain <code>dev-key</code> (or the value of <code>WAF_API_KEY</code>)
Port 8000 busy	<code>PORT=8010 ./scripts/setup_and_run.sh</code> (or stop the old process)
<code>pip install</code> slow	PyTorch is ~200 MB; only the first install downloads it
Windows: <code>python</code> not found	install Python 3.11+ from python.org and tick “Add to PATH”

10. Credits

- Team SecuraFormer — Transformer WAF (Smart India Hackathon) and this extension.
- Dataset: Prasad, A. & Chandra, S. (2023). *PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning*. Computers & Security. UCI ML Repository, <https://doi.org/10.24432/C5J92R> (CC BY 4.0). Only URL and label columns used.